文档

1. kafka官方文档：https://kafka.apache.org/documentation/
2. kafka docker ：https://kafka.apache.org/documentation/#docker
3. kafka docker 配置：https://github.com/apache/kafka/blob/trunk/docker/examples/README.md
4. kafka go lib：https://github.com/IBM/sarama/tree/main
5. kafka配置文件参考：https://github.com/apache/kafka/blob/3.7.0/config/kraft/server.properties
6. kafka用户名密码认证配置：https://kafka.apache.org/41/security/authentication-using-sasl/#authentication-using-saslplain

kafka

1. docker镜像

```
docker pull apache/kafka:3.7.0
```

2. 按默认配置运行kafka

```
docker run -p 9092:9092 apache/kafka:3.7.0
```

3. 使用文件输入提供配置
    1. 要求用户提供一个包含Kafka属性文件的本地文件夹路径，并使用Docker卷将其挂载到Docker容器中
    2. 将包含kafka属性文件的文件夹装载到docker容器中的/mnt/shared/config
    3. 例如：docker run --volume path/to/property/folder:/mnt/shared/config -p 9092:9092 apache/kafka:latest
4. 使用环境变量
    1. 通过env变量定义的Kafka属性将覆盖在文件输入和默认配置中定义的该属性的值
    2. 如果属性仅通过环境变量提供，则默认配置将被用户提供的属性所取代
    3. 构建服务器属性配置的环境键变量名时，可以按照以下步骤进行操作：
        1. 将"."替换为"_"
        2. 将"_"替换为"__"（双下划线）
        3. 将"-"替换为"___"（三个下划线）
        4. 在结果前面加上 "KAFKA"
        5. 举例说明：
            1. 对于 abc.def，使用 KAFKA_ABC_DEF
            2. 对于 abc-def，使用 KAFKA_ABC___DEF
            3. 对于 abc_def，使用 KAFKA_ABC__DEF
    4. 在为log4j属性文件提供配置时，应考虑以下几点：
    5. 通过环境变量提供的log4j属性将会追加到默认的属性文件中（与kafka捆绑在一起的log4j属性文件）

6. 可以通过设置 KAFKA_LOG4J_ROOT_LOGLEVEL 来设置 log4j.properties 和 tools-log4j.properties 中 log4j.rootLogger 的值
7. 可以通过将它们以逗号分隔的形式设置在 KAFKA_LOG4J_LOGGERS 环境变量中来添加 log4j 日志记录器到 log4j.properties 文件中
8. 示例：
    1. 假设向Docker容器提供了 KAFKA_LOG4J_LOGGERS='property1=value1,property2=value2' 环境变量。
    2. log4j.logger.property1=value1 和 log4j.logger.property2=value2 将被添加到Docker容器内部的 log4j.properties 文件中。
9. 常用于Kafka的环境变量可以通过环境变量进行提供，例如 CLUSTER_ID。
10. 可以使用命令 docker run --env CONFIG_NAME=CONFIG_VALUE -p 9092:9092 apache/kafka:latest 来为Docker容器提供环境变量。
11. 请注意，建议使用docker-compose文件来使用环境变量提供配置。

docker部署Kafka集群

集群配置说明

1. 服务器地址：192.168.239.164
2. Kafka集群节点描述：

	容器名称	访问端口	对应存储
节点1	kafka-1	29092	kafka1-data
节点2	kafka-2	39092	kafka2-data
节点3	kafka-3	49092	kafka3-data

获取Kafka运行用户ID

```
docker run --rm apache/kafka:3.7.0 id
```

```
nick@ubuntu-164:~/work/kafkaconf$ docker run --rm apache/kafka:3.7.0 id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
nick@ubuntu-164:~/work/kafkaconf$
```

为节点准备存储卷

1. 修改存储卷的所属用户和权限

```
docker run --rm -v kafka1-data:/data/kafka/logs quay.io/0voice/alpine:3.18 \
sh -c "chown -R 1000:1000 /data/kafka/logs && chmod -R 755 /data/kafka/logs"

docker run --rm -v kafka2-data:/data/kafka/logs quay.io/0voice/alpine:3.18 \
sh -c "chown -R 1000:1000 /data/kafka/logs && chmod -R 755 /data/kafka/logs"

docker run --rm -v kafka3-data:/data/kafka/logs quay.io/0voice/alpine:3.18 \
sh -c "chown -R 1000:1000 /data/kafka/logs && chmod -R 755 /data/kafka/logs"
```

准备配置文件

1. kafka_server_jaas.conf：为 SASL 认证提供用户凭证；该文件所有节点配置没有区别

```
# SASL/PLAIN认证的JAAS配置（监听器专属）：指定合法的用户名和密码，用于Broker间和客户端认证
KafkaServer {
    org.apache.kafka.common.security.plain.PlainLoginModule required
    username="admin"
    password="123456"

    user_admin="123456"
    user_alice="abcdefg";
};
```

```
1. username与password 用于当前节点连接到其他节点
2. user_<user>=<pwd> 表示创建一个用户并指定密码
3. 格式，不能包含#注释，最后一行必须以分号结尾
```

2. server.properties：Kafka服务配置；该文件节点之间存在节点ID和监听地址的区别

```
############################# Server Basics #############################
# 节点角色：同时作为Broker（消息服务）和Controller（集群管理），开启Kraft模式（无需ZooKeeper）
process.roles=broker,controller

# 节点唯一ID：集群内每个节点的ID必须唯一（1/2/3分别对应kafka-1/kafka-2/kafka-3）
node.id=1

# Controller集群的投票节点列表：格式为<node.id>@<主机名/IP>:<控制器端口>，需包含所有Controller节点
controller.quorum.voters=1@kafka-1:9093,2@kafka-2:9093,3@kafka-3:9093

############################# Listener配置 #############################
# 监听器列表：定义节点监听的地址和端口，格式为<监听器名称>://<主机名>:<端口>（:表示绑定所有网卡）
# SASL_PLAINTEXT：供客户端和Broker间通信的监听器（SASL认证+明文传输）
# CONTROLLER：供Controller节点间Raft协议通信的监听器（仅集群内部使用）
listeners=SASL_PLAINTEXT://:9092,CONTROLLER://kafka-1:9093,INTER_BROKER://kafka-1:19092

# Broker间通信使用的监听器名称：指定集群内Broker之间通信的监听器（需与listeners中的名称一致）
inter.broker.listener.name=INTER_BROKER

# 对外公告的监听器地址：客户端实际连接的地址，需填写可被外部访问的主机名/IP
# 注意：同一监听器名称可绑定多个地址（内网+外网），但不能包含节点ID（如1@）
advertised.listeners=INTER_BROKER://kafka-1:19092,SASL_PLAINTEXT://192.168.239.164:29092
```

```
# 控制器使用的监听器名称：指定Controller节点间通信的监听器（需与listeners中的CONTROLLER对应）
controller.listener.names=CONTROLLER

# 监听器名称与安全协议的映射表：将自定义监听器名称绑定到Kafka预定义的安全协议
# CONTROLLER:PLAINTEXT：控制器监听器使用明文协议（无认证，仅内部通信）
# PLAINTEXT:PLAINTEXT：备用监听器名称，映射到明文协议
# SSL:SSL：备用监听器名称，映射到SSL加密协议
# SASL_PLAINTEXT:SASL_PLAINTEXT：SASL认证+明文传输协议
# SASL_SSL:SASL_SSL：SASL认证+SSL加密协议
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,SSL:SSL,SASL_PLAINTE
XT:SASL_PLAINTEXT,SASL_SSL:SASL_SSL,INTER_BROKER:SASL_PLAINTEXT

############################## 安全认证配置 ##############################
# Broker间通信的SASL认证机制：指定使用PLAIN（用户名/密码）认证
sasl.mechanism.inter.broker.protocol=PLAIN

# 服务端启用的SASL认证机制列表：仅启用PLAIN机制（可多个，用逗号分隔）
sasl.enabled.mechanisms=PLAIN

############################## 网络线程配置 ##############################
# 网络处理线程数：用于接收网络请求和发送响应的线程数（建议3-5，根据CPU核心数调整）
num.network.threads=3
# IO处理线程数：用于处理请求（如磁盘IO）的线程数（建议为CPU核心数的2倍）
num.io.threads=8
# 套接字发送缓冲区大小：SO_SNDBUF的大小，单位字节（默认100KB）
socket.send.buffer.bytes=102400
# 套接字接收缓冲区大小：SO_RCVBUF的大小，单位字节（默认100KB）
socket.receive.buffer.bytes=102400
# 最大请求大小：服务端接受的最大请求字节数（防止OOM，默认100MB）
socket.request.max.bytes=104857600

############################## 日志存储配置 ##############################
# 日志数据存储目录：Kafka的消息日志、元数据等数据的存储路径
# 注意：/tmp为临时目录，重启服务器后数据丢失，生产环境需改为持久化目录（如/data/kafka/logs）
log.dirs=/data/kafka/logs
# 默认分区数：新建主题时的默认分区数（建议设为集群节点数，如3，提升并行性）
num.partitions=3
# 每个数据目录的恢复线程数：用于日志恢复和刷盘的线程数（RAID阵列可适当增加）
num.recovery.threads.per.data.dir=1

############################## 内部主题配置 ##############################
# 消费者偏移量主题的副本因子：__consumer_offsets主题的分区副本数（建议与集群节点数一致，保证高可用）
offsets.topic.replication.factor=3
# 事务状态日志主题的副本因子：__transaction_state主题的分区副本数（建议与集群节点数一致）
transaction.state.log.replication.factor=3
# 事务状态日志的最小同步副本数：必须小于等于副本因子（建议设为2，保证数据可靠性）
transaction.state.log.min.isr=2

############################## 日志刷新策略 ##############################
# 消息会立即写入文件系统，但默认情况下，我们只使用fsync（）延迟同步操作系统缓存。以下配置控制将数据刷新到磁盘。这里有一些重要的权衡：
# 1.持久性：如果不使用复制，未清理的数据可能会丢失。
```

```
# 2.延迟：当刷新发生时，非常大的刷新间隔可能会导致延迟峰值，因为将有大量数据需要刷新。
# 3.吞吐量：刷新通常是最昂贵的操作，小的刷新间隔可能会导致过多的寻道。
# 下面的设置允许配置刷新策略，以便在一段时间后或每N条消息（或两者兼有）刷新数据。这可以全局完成，并在每个主题的基础上覆盖

# 强制将数据刷新到磁盘之前要接受的消息数
#log.flush.interval.messages=10000

# 在我们强制刷新之前，消息可以在日志中停留的最长时间
#log.flush.interval.ms=1000

############################# 日志保留策略 #############################
# 日志保留时间：日志文件的最大保留时长（默认7天，168小时）
log.retention.hours=168
# 日志段大小：单个日志文件的最大大小（默认1GB，达到后新建日志段）
log.segment.bytes=1073741824
# 日志保留检查间隔：定期检查日志是否需要删除的时间间隔（默认5分钟，300000毫秒）
log.retention.check.interval.ms=300000
```

3. server.properties节点间配置的区别

```
# kafka-1
node.id=1
listeners=SASL_PLAINTEXT://:9092,CONTROLLER://kafka-1:9093,INTER_BROKER://kafka-1:19092
advertised.listeners=INTER_BROKER://kafka-1:19092,SASL_PLAINTEXT://192.168.239.164:29092

# kafka-2
node.id=2
listeners=SASL_PLAINTEXT://:9092,CONTROLLER://kafka-2:9093,INTER_BROKER://kafka-2:19092
advertised.listeners=INTER_BROKER://kafka-2:19092,SASL_PLAINTEXT://192.168.239.164:39092

# kafka-3
node.id=3
listeners=SASL_PLAINTEXT://:9092,CONTROLLER://kafka-3:9093,INTER_BROKER://kafka-3:19092
advertised.listeners=INTER_BROKER://kafka-3:19092,SASL_PLAINTEXT://192.168.239.164:49092
```

创建网络

```
docker network create kafka-net
```

设置集群ID到环境变量

```
# 设置集群ID
KAFKA_CLUSTER_ID=$(uuidgen)
# 输出集群ID
echo $KAFKA_CLUSTER_ID
```

启动集群节点

```
# 节点1
```

```
docker run -d --name kafka-1 -p 29092:9092 --network kafka-net  \
-e CLUSTER_ID=$KAFKA_CLUSTER_ID \
-e KAFKA_OPTS="-Djava.security.auth.login.config=/mnt/shared/config/kafka_server_jaas.conf"
\
-v kafka1-data:/data/kafka/logs \
-v /home/nick/work/kafkaconf/kafka1:/mnt/shared/config \
apache/kafka:3.7.0

# 节点2
docker run -d --name kafka-2 -p 39092:9092 --network kafka-net  \
-e CLUSTER_ID=$KAFKA_CLUSTER_ID \
-e KAFKA_OPTS="-Djava.security.auth.login.config=/mnt/shared/config/kafka_server_jaas.conf"
\
-v kafka2-data:/data/kafka/logs \
-v /home/nick/work/kafkaconf/kafka2:/mnt/shared/config \
apache/kafka:3.7.0

# 节点3
docker run -d --name kafka-3 -p 49092:9092 --network kafka-net  \
-e CLUSTER_ID=$KAFKA_CLUSTER_ID \
-e KAFKA_OPTS="-Djava.security.auth.login.config=/mnt/shared/config/kafka_server_jaas.conf"
\
-v kafka3-data:/data/kafka/logs \
-v /home/nick/work/kafkaconf/kafka3:/mnt/shared/config \
apache/kafka:3.7.0
```

验证集群

配置命令行客户端访问凭据

1. 与容器交互，进入容器环境

```
docker exec -it kafka-1 bash
# 进入用户目录
cd
```

2. vi client.properties 新建client.properties文件
3. 配置命令行客户端访问凭证，client.properties

```
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required
username="admin" password="123456";
```

检查集群状态

```
/opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server 192.168.239.164:29092 --
command-config client.properties
```

```
36fc32ca8151:~$ /opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server 192.168.239.164:29092 --command-config client.properties
192.168.239.164:29092 (id: 1 rack: null) -> (
        Produce(0): 0 to 10 [usable: 10],
```

docker部署单节点Kafka集群

获取Kafka运行用户ID

```
docker run --rm apache/kafka:3.7.0 id
```

```
nick@ubuntu-164:~/work/kafkaconf$ docker run --rm apache/kafka:3.7.0 id
uid=1000(appuser) gid=1000(appuser) groups=1000(appuser)
nick@ubuntu-164:~/work/kafkaconf$
```

为节点准备存储卷

```
docker run --rm -v kafka-single-data:/data/kafka/logs quay.io/0voice/alpine:3.18 \
sh -c "chown -R 1000:1000 /data/kafka/logs && chmod -R 755 /data/kafka/logs"
```

准备配置文件

1. kafka_server_jaas.conf：为 SASL 认证提供用户凭证；

```
# SASL/PLAIN认证的JAAS配置（监听器专属）：指定合法的用户名和密码，用于Broker间和客户端认证
KafkaServer {
    org.apache.kafka.common.security.plain.PlainLoginModule required
    username="admin"
    password="123456"

    user_admin="123456"
    user_alice="abcdefg";
};
```

2. server.properties：Kafka服务配置

```
process.roles=broker,controller
node.id=1
controller.quorum.voters=1@kafka-single:9093
listeners=SASL_PLAINTEXT://:9092,CONTROLLER://kafka-single:9093,INTER_BROKER://kafka-single:19092
inter.broker.listener.name=INTER_BROKER
advertised.listeners=INTER_BROKER://kafka-single:19092,SASL_PLAINTEXT://192.168.239.164:59092
controller.listener.names=CONTROLLER
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,SSL:SSL,SASL_PLAINTEXT:SASL_PLAINTEXT,SASL_SSL:SASL_SSL,INTER_BROKER:SASL_PLAINTEXT
sasl.mechanism.inter.broker.protocol=PLAIN
sasl.enabled.mechanisms=PLAIN
num.network.threads=3
num.io.threads=8
socket.send.buffer.bytes=102400
```

```
socket.receive.buffer.bytes=102400
socket.request.max.bytes=104857600
log.dirs=/data/kafka/logs
num.partitions=1
num.recovery.threads.per.data.dir=1
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
log.retention.hours=168
log.segment.bytes=1073741824
log.retention.check.interval.ms=300000
```

创建网络

```
docker network create kafka-net
```

设置集群ID到环境变量

```
# 设置集群ID
KAFKA_CLUSTER_ID=$(uuidgen)
# 输出集群ID
echo $KAFKA_CLUSTER_ID
```

启动节点

```
docker run -d --name kafka-single -p 59092:9092 --network kafka-net  \
-e CLUSTER_ID=$KAFKA_CLUSTER_ID \
-e KAFKA_OPTS="-Djava.security.auth.login.config=/mnt/shared/config/kafka_server_jaas.conf"
\
-v kafka-single-data:/data/kafka/logs \
-v /home/nick/work/kafkaconf/kafkasingle:/mnt/shared/config \
apache/kafka:3.7.0
```

验证集群

配置命令行客户端访问凭据

1. 与容器交互，进入容器环境

```
docker exec -it kafka-single bash
# 进入用户目录
cd
```

2. vi client.properties 新建client.properties文件
3. 配置命令行客户端访问凭证，client.properties

```
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required
username="admin" password="123456";
```

检查集群状态

```
/opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server 192.168.239.164:59092 --
command-config client.properties
```

```
a4e46055c3fd:~$ /opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server 192.168.239.164:59092 --command-config client.properties
192.168.239.164:59092 (id: 1 rack: null) -> (
        Produce(0): 0 to 10 [usable: 10],
        Fetch(1): 0 to 16 [usable: 16],
        ListOffsets(2): 0 to 8 [usable: 8],
```

Kafka命令行

配置命令行客户端访问凭据

1. 与容器交互，进入容器环境

```
docker exec -it kafka-1 bash
```

2. vi client.properties 新建client.properties文件
3. 配置命令行客户端访问凭证，client.properties

```
security.protocol=SASL_PLAINTEXT
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required
username="admin" password="123456";
```

Topic

2. 查看所有Topic

```
./kafka-topics.sh --bootstrap-server <kafka-server> --list --command-config
client.properties
```

3. 查看Topic详情

```
./kafka-topics.sh --bootstrap-server <kafka-server> --describe --topic <topic-name>  --
command-config client.properties
```

4. 查看Topic数据

```
./kafka-console-consumer.sh --bootstrap-server <kafka-server> --topic <topic-name> --from-
beginning --consumer.config client.properties
```

5. 为主题添加分区

```
./kafka-topics.sh --alter --bootstrap-server <kafka-bootstrap-servers> --topic <topic-name>
--partitions <new-partition-count> --command-config client.properties
```

6. 查看分区数据

```
./kafka-console-consumer.sh --bootstrap-server <kafka-bootstrap-servers> --topic <topic-
name>  --partition <partition_id>  --from-beginning --consumer.config client.properties
```